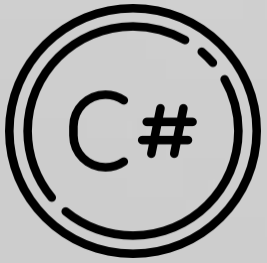
Func vs Action vs Predicate



Created by Abdelrahman Ashraf Abdelrahman.ashrf1@gmail.com

They are **built-in generic delegates** that Microsoft provides so you don't have to write your own delegate types.

1 Func

- Represents a **method that returns a value**.
- Can have **0 to 16 input parameters**.
- The **last type parameter** is always the **return type**.

```
// Func<int, int, int> means: takes (int, int) → returns int
Func<int, int, int> add = (a, b) => a + b;

int result = add(5, 3);  // result = 8
```

Created by Abdelrahman Ashraf Abdelrahman.ashrf1@gmail.com

2 Action

- **Represents a method that does not return anything (void).**
- **Can have 0 to 16 input parameters.**

```
// Action<string> means: takes (string) → returns void
Action<string> printMessage = (msg) => Console.WriteLine(msg);

printMessage("Hello Action!");  // Output: Hello Action!
```

Example with no parameters:

```
Action sayHi = () => Console.WriteLine("Hi there!");
sayHi();  // Output: Hi there!
```

Created by Abdelrahman Ashraf Abdelrahman.ashrf1@gmail.com

3 Predicate

- Represents a method that **takes 1 input parameter** and **returns a bool**.
- Shortcut for Func<T, bool>.

```
// Predicate<int> means: takes int → returns bool
Predicate<int> isEven = (num) => num % 2 == 0;

Console.WriteLine(isEven(4));  // True
Console.WriteLine(isEven(5));  // False
```

Equivalent using Func:

```
Func<int, bool> isEvenFunc = (num) => num % 2 == 0;
```

Created by Abdelrahman Ashraf Abdelrahman.ashrf1@gmail.com

Summary

Delegate	Returns	Parameters
Func<T1, ..., TResult>	TResult	0–16 parameters
Action<T1, ...>	void	0–16 parameters
Predicate<T>	bool	Exactly 1 parameter

✅ **When to use them?**

- Func → when you need a return value.
- Action → when you just want to execute code with no return.
- Predicate → when you need a bool condition check.

Created by Abdelrahman Ashraf Abdelrahman.ashrf1@gmail.com